



TECNOLÓGICO
NACIONAL DE MÉXICO



TECNOLÓGICO NACIONAL DE
MÉXICO CAMPUS CULIACÁN

TAREA BONUS MODULO II

Temas de Inteligencia Artificial

Rodriguez Lopez Said Armando
Santacruz Maldonado Oscar Said

Zuriel Dathan Mora Felix

Ingeniería en Sistemas computacionales
9no Semestre

9:00 - 10:00

09/10/2026

Índice

1. Introducción.....	2
2. Problema de programación de trabajos (JSSP).....	3
3. Problema de las N Reinas	6
4. Árbol de expansión mínima (MST)	8
5. Problema del agente viajero (TSP)	10
6. Conclusiones	11
7. Referencias	12

1. Introducción

La optimización combinatoria se trata de problemas donde hay que escoger la mejor opción de entre un espacio de soluciones muy grande, que aunque son finitas pueden ser demasiadas para revisarlas una por una (“Combinatorial Optimization”, s.f.). Cosas como acomodar tareas en un taller o encontrar la ruta más corta para repartir algo entran en esta área.

Para este trabajo se analizarán 4 problemas, el de programación de trabajos o JSSP, las N Reinas, el árbol de expansión mínima (MST) y el agente viajero (TSP). Aunque los cuatro son de optimización cada uno es de un tipo distinto, el JSSP es de planificar tareas en el tiempo, las N Reinas es más de cumplir restricciones, el MST es de grafos y el TSP de rutas.

Algo que se repite en todos es que antes de hacer cualquier algoritmo hay que pensar cómo se van a guardar los datos en la computadora, porque de eso depende lo que se puede revisar después (GeeksforGeeks, 2025). Por eso en cada problema además de explicar en qué consiste se propondrá una estructura de datos con la que se podría trabajar a la hora de querer representar el problema.

2. Problema de programación de trabajos (JSSP)

El JSSP (Job Shop Scheduling Problem) o problema de programación de trabajos en taller es sobre decidir en qué orden cada máquina va a hacer las operaciones de varios trabajos, para que todo se acabe lo antes posible. Según Cozzolino et al. (2021) es de los modelos de programación más estudiados.

El ejemplo mas común es imaginar un taller mecánico con un torno, una fresadora y un taladro. Cada pieza que llega al taller tiene que pasar por las tres pero no todas en el mismo orden, una empieza en el torno y otra en el taladro, y si dos piezas necesitan la misma máquina al mismo tiempo una se tiene que esperar. El problema consiste en decidir cuál espera.

Lo que define una instancia es:

- n trabajos $J_1, J_2, \dots, J_n$ y m máquinas $M_1, M_2, \dots, M_m$.
- cada trabajo J_i tiene sus operaciones $O_{i1}, O_{i2}, \dots, O_{im}$ en un orden fijo, y cada operación O_{ik} va en una máquina en específico y tarda un tiempo p_{ik} .

Las reglas son que las operaciones de un trabajo van en su orden (O_{i2} no empieza hasta que acaba O_{i1}), una máquina solo hace una operación a la vez y si una operación empieza ya no se puede pausar. Cada trabajo pasa una vez por cada máquina y todos están listos desde el tiempo 0.

Para representar una instancia podemos usar dos matrices de $n \times m$, una para las máquinas y otra para los tiempos. Las filas son los trabajos y las columnas el número de operación, entonces $M[i][k]$ dice en qué máquina va la operación k del trabajo i mientras que $P[i][k]$ nos dice cuánto tarda. Con 3 trabajos y 3 máquinas quedaría así:

Matriz de máquinas M (orden):

Trabajo	Operación 1	Operación 2	Operación 3
J1	M1	M2	M3
J2	M1	M3	M2
J3	M2	M3	M1

Matriz de tiempos P (tiempo):

Trabajo	Operación 1	Operación 2	Operación 3
J1	3	2	2
J2	2	1	4
J3	4	3	1

Las dos matrices se tienen que leer juntas porque una sola no alcanza, en la de máquinas no están los tiempos y en la de tiempos no se sabe a qué máquina va cada uno. Por eso conviene ver cómo se relacionan, si se ponen las dos en una sola tabla cada casilla $[i][k]$ queda con la máquina $M[i][k]$ y entre paréntesis el tiempo $P[i][k]$ que tarda esa operación:

Tenemos que tener en cuenta que no basta con sumar los tiempos, porque mientras una máquina trabaja otra puede estar trabajando también. Hay que ir viendo con la matriz de tiempos P cuándo puede empezar cada operación, que es cuando ya acabó la anterior de su trabajo y la máquina ya está libre.

Relación entre las matrices M y P:

Trabajo	Operación 1	Operación 2	Operación 3
J1	M1 (3)	M2 (2)	M3 (2)
J2	M1 (2)	M3 (1)	M2 (4)
J3	M2 (4)	M3 (3)	M1 (1)

Para entenderlo mejor podemos seguir al trabajo J2 en esta tabla. Su fila dice M1 (2), M3 (1) y M2 (4), lo que significa que primero va a la máquina M1 y ahí tarda 2 unidades de tiempo, cuando termina pasa a M3 donde tarda 1, y al final va a M2 donde tarda 4. Si solo tuviéramos la matriz de máquinas sabríamos que el recorrido es $M1 \rightarrow M3 \rightarrow M2$ pero no cuánto dura cada paso, y si solo tuviéramos la de tiempos tendríamos los números 2, 1 y 4 sin saber en qué máquina pasa cada uno. Por ejemplo, el 4 que está en $P[2][3]$ solo tiene sentido cuando vemos en $M[2][3]$ que le toca a la máquina M2.

Si se suman sus tiempos, J2 necesita $2 + 1 + 4 = 7$ unidades, pero eso no quiere decir que vaya a terminar en el tiempo 7. En la tabla se ve que J1 también empieza en M1, así que los dos no pueden arrancar al mismo tiempo. Si J1 entra primero ocupa M1 de 0 a 3 y J2 tiene que esperar hasta el tiempo 3 para empezar, y si J2 entra primero entonces el que espera es J1. Decidir quién espera en cada máquina es justo lo que trata de resolver el problema.

3. Problema de las N Reinas

Este se refiere a un problema de ajedrez, consiste en que hay que poner N reinas en un tablero de $N \times N$ sin que ninguna se pueda comer a otra. Originalmente era con 8 reinas, así lo publicó Max Bezzel en una revista de ajedrez en 1848, y luego se pasó a cualquier tamaño de tablero ("Eight Queens Puzzle", s.f.). La reina se mueve en línea recta para cualquier lado y también en diagonal, así que no puede haber dos reinas en la misma fila, ni en la misma columna, ni en la misma diagonal.

Si una reina está en (i_1, j_1) y otra en (i_2, j_2) quedan en la misma diagonal cuando $|i_1 - i_2| = |j_1 - j_2|$, porque al moverse en diagonal se avanza lo mismo en filas que en columnas. Otra forma de verlo es que en una diagonal $i + j$ siempre da lo mismo y en la otra lo que no cambia es $i - j$.

Como ya es un tablero, la estructura que se propone es una matriz booleana de $N \times N$, que es básicamente el tablero con unos y ceros, 1 si hay reina en esa casilla y 0 si no. En este de 4×4 hay dos reinas, una en $(1, 2)$ y otra en $(3, 4)$:

Fila	Columna 1	Columna 2	Columna 3	Columna 4
1	0	1	0	0
2	0	0	0	0
3	0	0	0	1
4	0	0	0	0

Estas dos se atacan porque $|1 - 3| = 2$ y $|2 - 4| = 2$, o sea que están en diagonal.

No siempre hay solución, con 2 o 3 reinas no se puede, y en el tablero normal de 8×8 hay 92 formas de acomodarlas (Weisstein, s.f.-c). Es un problema que se usa bastante para enseñar backtracking (GeeksforGeeks, 2026).

Una solución del tablero de 4×4 se puede representar con la misma matriz. Por ejemplo, poniendo las reinas en (1, 2), (2, 4), (3, 1) y (4, 3) queda así:

Fila	Columna 1	Columna 2	Columna 3	Columna 4
1	0	1	0	0
2	0	0	0	1
3	1	0	0	0
4	0	0	1	0

Esta matriz sí es una solución porque ninguna reina se puede comer a otra. Horizontalmente no se chocan porque cada fila tiene un solo 1, y verticalmente tampoco porque cada columna también tiene un solo 1. Para las diagonales se usa la misma regla de antes, comparando todas las parejas de reinas ninguna tiene la diferencia de filas igual a la diferencia de columnas, por ejemplo entre (1, 2) y (2, 4) las filas cambian 1 y las columnas 2, y entre (2, 4) y (3, 1) las filas cambian 1 y las columnas 3. También se puede comprobar con las sumas $i + j$, que dan 3, 6, 4 y 7, y con las restas $i - j$, que dan -1 , -2 , 2 y 1, como ningún valor se repite no hay dos reinas en la misma diagonal. Como se cumplen las tres condiciones, que no se repita fila, ni columna, ni diagonal, la matriz representa una solución válida.

4. Árbol de expansión mínima (MST)

El árbol de expansión mínima (Minimum Spanning Tree, MST) es un problema de grafos en el que, teniendo un grafo conexo y no dirigido donde cada arista tiene un peso, se buscan las aristas que conecten todos los vértices sin formar ningún ciclo y con la menor suma de pesos posible (Weisstein, s.f.-b). Se le dice árbol porque el resultado no tiene ciclos, de expansión porque llega a todos los vértices y mínimo porque es el de menor costo.

Por ejemplo, supongamos que una empresa quiere llevar fibra óptica a cinco colonias. Sabe cuánto cuesta tender cable entre cada par de colonias y quiere que todas queden conectadas gastando lo menos posible, aunque sea de forma indirecta, si A ya llega a C pasando por B no tiene caso poner un cable de A a C. Eso es lo que resuelve el árbol de expansión mínima, y de hecho así empezó, Borůvka en 1926 lo trabajó para una red eléctrica en Moravia y después salieron otros métodos como los de Kruskal en 1956 y Prim en 1957.

Dicho con grafos, las colonias son vértices, los cables posibles son aristas y cada arista tiene un peso que es su costo. El grafo $G = (V, E)$ tiene que ser no dirigido y conexo, y lo que se busca es un conjunto de aristas que una todos los vértices, que no tenga ciclos y que sume el menor peso (Weisstein, s.f.-b). Un árbol así siempre queda con $|V| - 1$ aristas, y si hay pesos repetidos puede que haya más de un árbol con el mismo costo mínimo. La estructura que se propone es una matriz de adyacencia de $V \times V$ (Weisstein y Sauras-Altuzarra, s.f.), filas y columnas son los vértices y en cada celda va el peso de la arista entre ellos, un guion si no hay arista y 0 en la diagonal. En la siguiente tabla podemos ver un ejemplo de posibles conexiones:

	A	B	C	D	E
A	0	2	3	-	-
B	2	0	1	4	-
C	3	1	0	5	6
D	-	4	5	0	7
E	-	-	6	7	0

Para ver cuánto cuesta unir B con D se busca la celda [B][D] y vale 4, entre A y D hay guion entonces no hay cable directo posible. La matriz sale simétrica porque el grafo no tiene dirección, la mitad de arriba es igual a la de abajo. Si alguien propone un árbol se puede revisar con la misma tabla, que cada arista elegida tenga valor, que no se cierre ningún ciclo y el costo total es sumar esas celdas.

5. Problema del agente viajero (TSP)

En el problema del agente viajero (Traveling Salesman Problem, TSP) un vendedor tiene que pasar por varias ciudades, cada una solo una vez, y al final volver a la ciudad de donde salió. Se conocen las distancias entre todas y la pregunta a resolver es cuál es el recorrido más corto. Parece sencillo pero es de los problemas más estudiados (Yu, 2014).

Si se ve como grafo, las ciudades son vértices y todas están conectadas entre sí, cada arista tiene un costo $c(i, j)$ y lo que se busca es un ciclo hamiltoniano de costo mínimo. Se llama así por Hamilton, que en 1857 hizo un juego de recorrer los vértices de un dodecaedro pasando una vez por cada uno (Weisstein, s.f.-a). Hay dos versiones, cuando ir de i a j cuesta lo mismo que de j a i es el TSP simétrico, y cuando no, como con calles de un solo sentido o subidas, es el asimétrico.

Para representarlo se propone una matriz de distancias D de $n \times n$, donde $D[i][j]$ es lo que cuesta ir de la ciudad i a la j y la diagonal es 0. Sirve para las dos versiones, en la simétrica la matriz queda simétrica y en la asimétrica no, pero se guarda igual. Como todas las ciudades están conectadas no queda ninguna celda vacía. Con cuatro ciudades:

	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Un recorrido es la lista de ciudades en el orden en que se visitan y cada ciudad tiene que salir una sola vez. Su costo es la suma de las celdas de cada tramo más la del regreso, por ejemplo $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$:

$$D[A][B] + D[B][D] + D[D][C] + D[C][A] = 10 + 25 + 30 + 15 = 80$$

6. Conclusiones

Los cuatro problemas se pueden representar con matrices, aunque no en todos queda igual de directo. En el problema de programación de trabajos una sola matriz no alcanza porque cada operación tiene dos datos, la máquina y el tiempo, por eso son dos. En el problema de las N Reinas pasa lo contrario, la matriz es literalmente el tablero.

Problema	Tipo de problema	Matriz usada	Tamaño	Qué guarda cada celda
Problema de programación de trabajos	Planificación	Máquinas y tiempos	$n \times m$ (dos matrices)	Máquina y duración de cada operación
Problema de las N Reinas	Satisfacción de restricciones	Tablero booleano	$N \times N$	1 si hay reina, 0 si no
Árbol de expansión mínima	Optimización en grafos	Adyacencia	$V \times V$	Peso de la arista o -
Problema del agente viajero	Rutas	Distancias	$n \times n$	Costo de viajar de i a j

En el árbol de expansión mínima y en el problema del agente viajero la matriz es casi la misma, filas y columnas son puntos y la celda es lo que cuesta unirlos, la diferencia es que en el agente viajero todas las ciudades están conectadas y en el árbol de expansión mínima puede haber guiones. Por eso creo que en el problema del agente viajero la matriz se aprovecha mejor, en un árbol de expansión mínima con pocas conexiones buena parte de la tabla quedaría con guiones. Algo parecido pasa con el problema de las N Reinas, en un tablero grande casi todo son ceros porque nada más hay N unos.

Lo que sí tienen en común es que se pueden revisar a mano, se dibuja la tabla en una hoja y se comprueban las restricciones viendo filas y columnas, lo cual ayuda a entender el problema antes de pensar en algún algoritmo.

7. Referencias

Cozzolino, C., Christiansen, B., Vallejos, E., Sorce, M., & Visk, J. (2021, 15 de diciembre). *Job shop scheduling*. Cornell University Computational Optimization Open Textbook.

https://optimization.cbe.cornell.edu/index.php?title=Job_shop_scheduling

GeeksforGeeks. (2025, 12 de julio). *Introduction to data structures*.

<https://www.geeksforgeeks.org/dsa/introduction-to-data-structures/>

GeeksforGeeks. (2026, 17 de agosto). *N Queen problem*. <https://www.geeksforgeeks.org/dsa/n-queen-problem-backtracking-3/>

Weisstein, E. W. (s.f.-a). *Icosian game*. MathWorld—A Wolfram Web Resource.

<https://mathworld.wolfram.com/IcosianGame.html>

Weisstein, E. W. (s.f.-b). *Minimum spanning tree*. MathWorld—A Wolfram Web Resource.

<https://mathworld.wolfram.com/MinimumSpanningTree.html>

Weisstein, E. W. (s.f.-c). *Queens problem*. MathWorld—A Wolfram Web Resource.

<https://mathworld.wolfram.com/QueensProblem.html>

Weisstein, E. W., & Sauras-Altuzarra, L. (s.f.). *Adjacency matrix*. MathWorld—A Wolfram Web Resource.

<https://mathworld.wolfram.com/AdjacencyMatrix.html>

Yu, J. (2014). *Traveling salesman problem*. Cornell University Computational Optimization Open

Textbook. https://optimization.cbe.cornell.edu/index.php/Traveling_salesman_problem

Combinatorial optimization. (s.f.). En *Wikipedia*. Recuperado el 9 de octubre de 2026, de

https://en.wikipedia.org/wiki/Combinatorial_optimization

Eight queens puzzle. (s.f.). En *Wikipedia*. Recuperado el 9 de octubre de 2026, de

https://en.wikipedia.org/wiki/Eight_queens_puzzle